

Dijkstra's Algorithm

Dijkstra's algorithm

Dijkstra's algorithm - is a solution to the single-source shortest path problem in graph theory.

Works on both directed and undirected graphs. However, all edges must have nonnegative weights.

Input: Weighted graph $G=\{E,V\}$ and source vertex $v \in V$, such that all edge weights are nonnegative

Output: Lengths of shortest paths (or the shortest paths themselves) from a given source vertex $v \in V$ to all other vertices

Approach

- The algorithm computes for each vertex u the **distance** to u from the start vertex v , that is, the weight of a shortest path between v and u .
- the algorithm keeps track of the set of vertices for which the distance has been computed, called the **cloud** C
- Every vertex has a label D associated with it. For any vertex u , $D[u]$ stores an approximation of the distance between v and u . The algorithm will update a $D[u]$ value when it finds a shorter path from v to u .
- When a vertex u is added to the cloud, its label $D[u]$ is equal to the actual (final) distance between the starting vertex v and vertex u .

Dijkstra pseudocode

Dijkstra(v1, v2):

for each vertex v: // Initialization

v's distance := infinity.

v's previous := none.

v1's distance := 0.

List := {all vertices}.

while List is not empty:

v := remove List vertex with minimum distance.

mark v as known.

for each unknown neighbor n of v:

dist := v's distance + edge (v, n)'s weight.

if dist is smaller than n's distance:

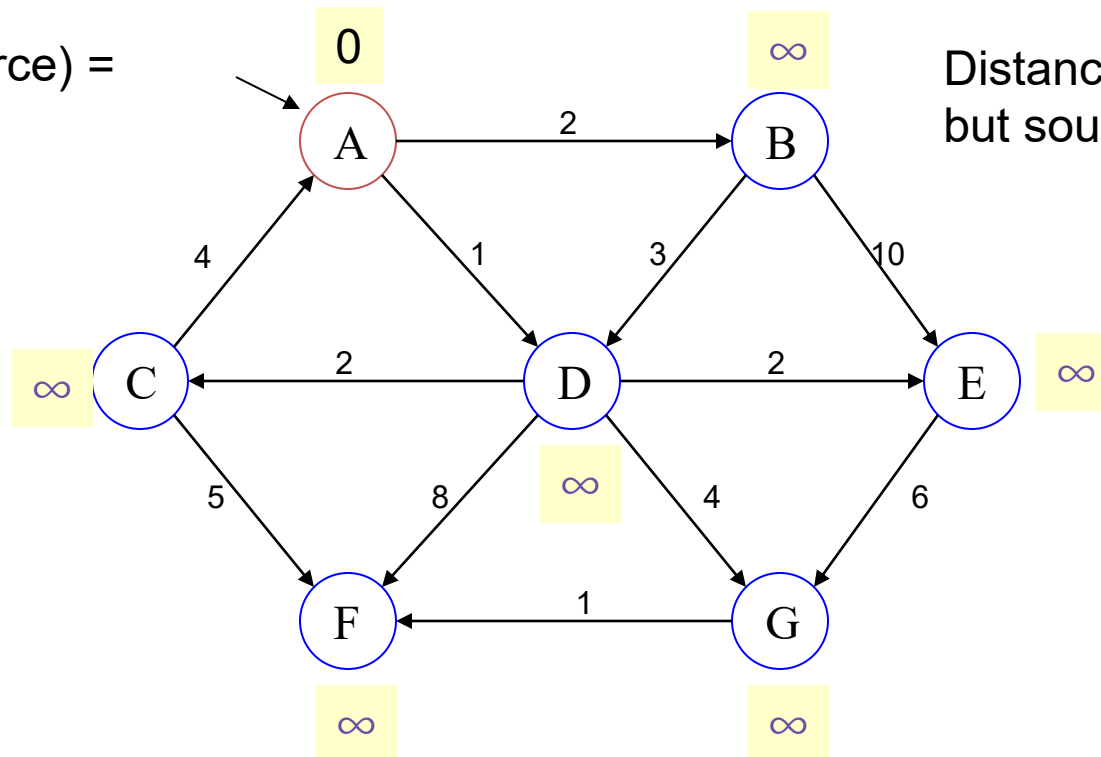
n's distance := dist.

n's previous := v.

*reconstruct path from v2 back to v1,
following previous pointers.*

Example: Initialization

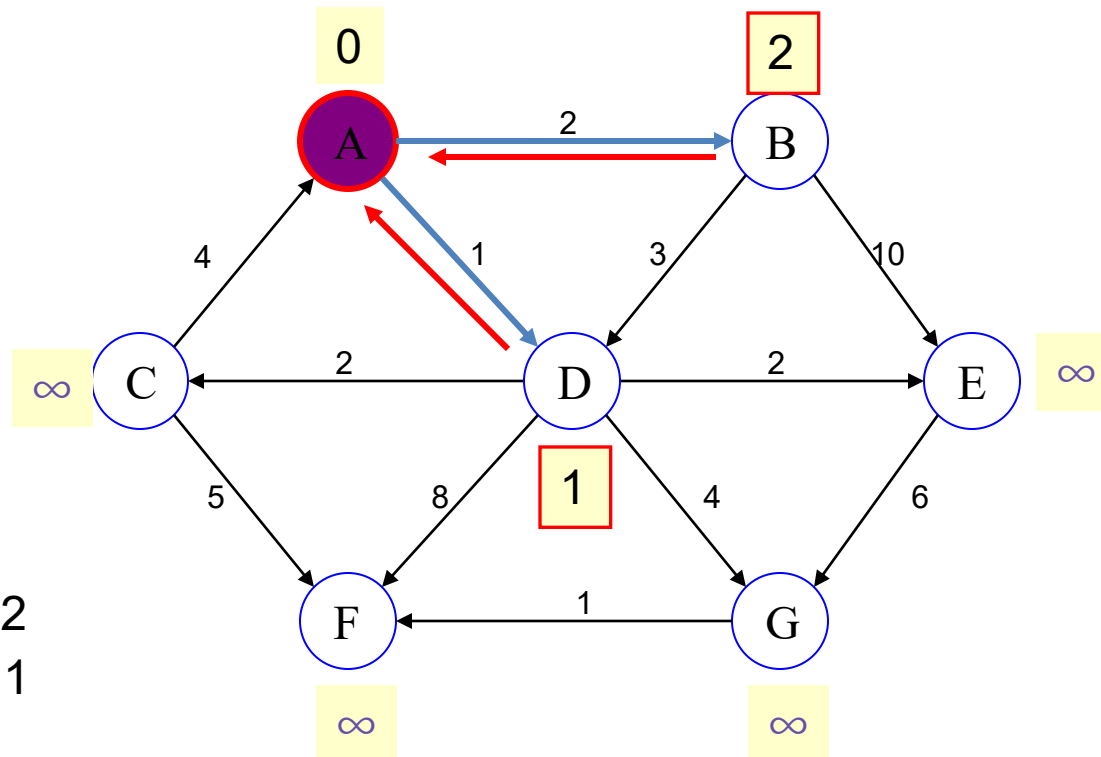
Distance(source) =
0



Distance (all vertices
but source) = ∞

Pick vertex in List with minimum distance.

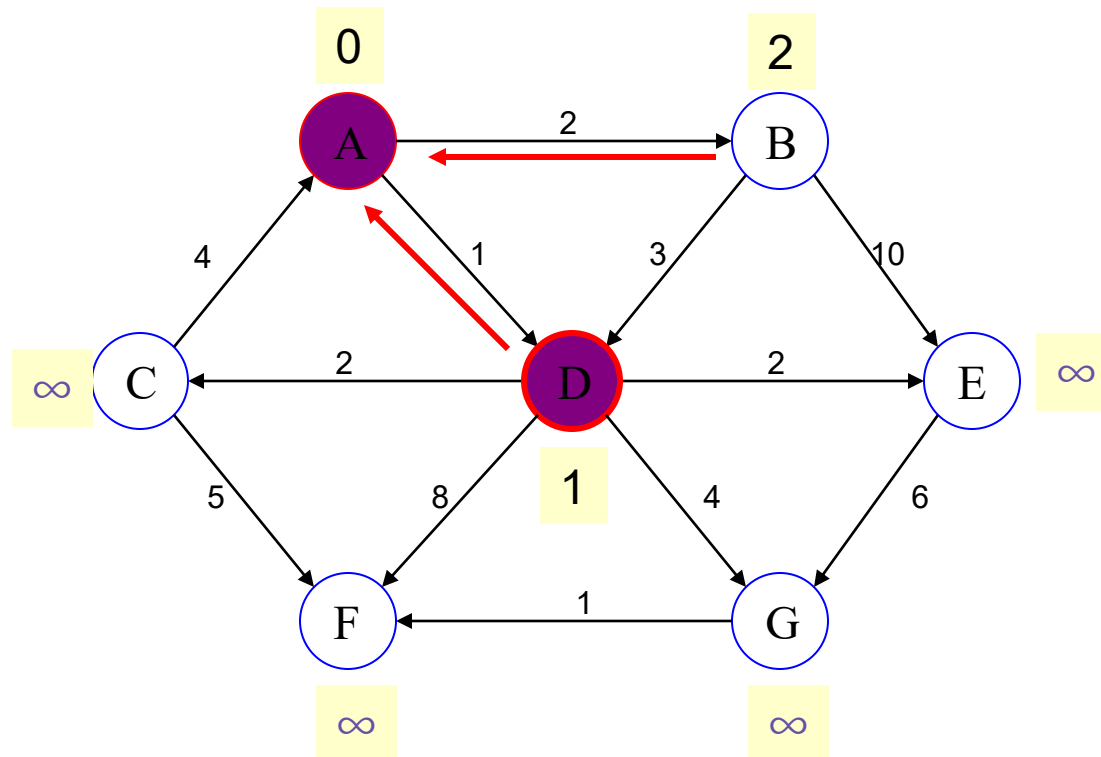
Example: Update neighbors' distance



Distance(B) = 2

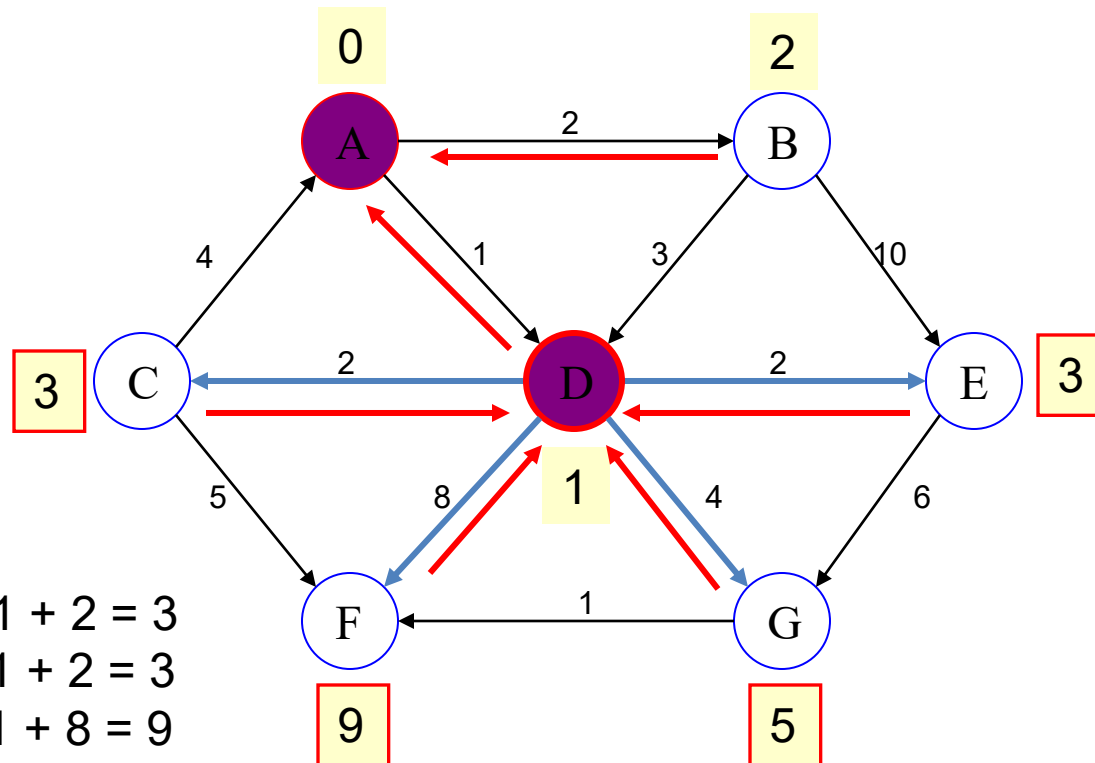
Distance(D) = 1

Example: Remove vertex with minimum distance



Pick vertex in List with minimum distance, i.e., D

Example: Update neighbors



Distance(C) = 1 + 2 = 3

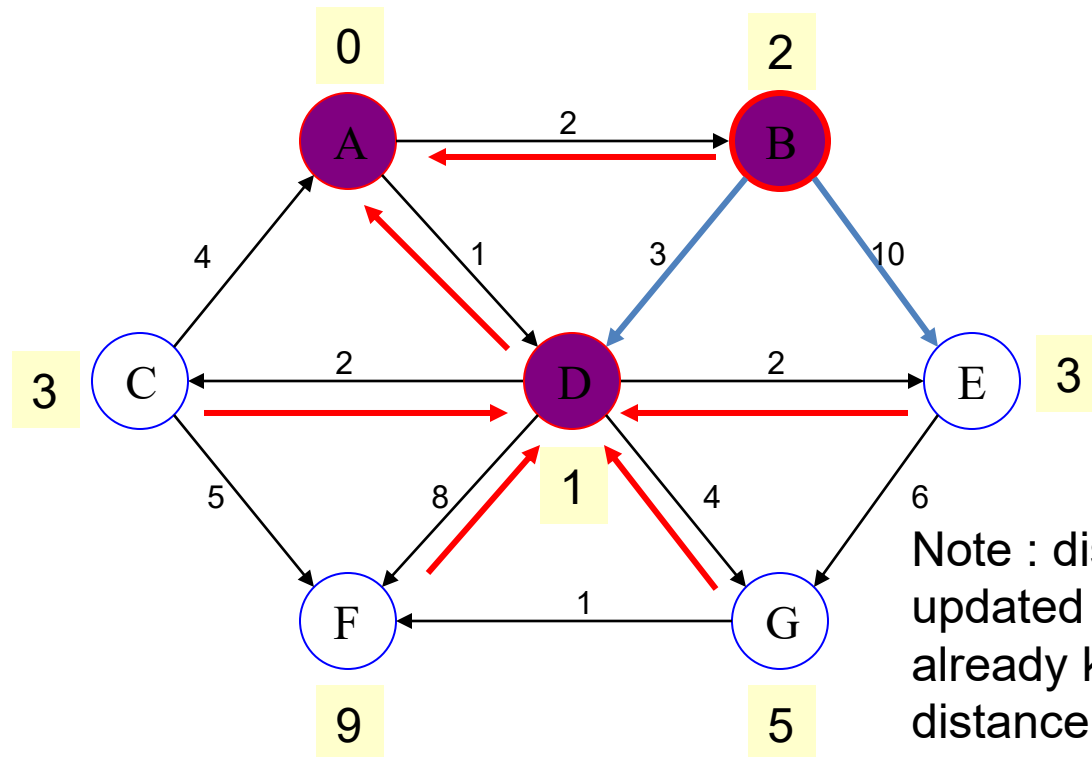
Distance(E) = 1 + 2 = 3

Distance(F) = 1 + 8 = 9

Distance(G) = 1 + 4 = 5

Example: Continued...

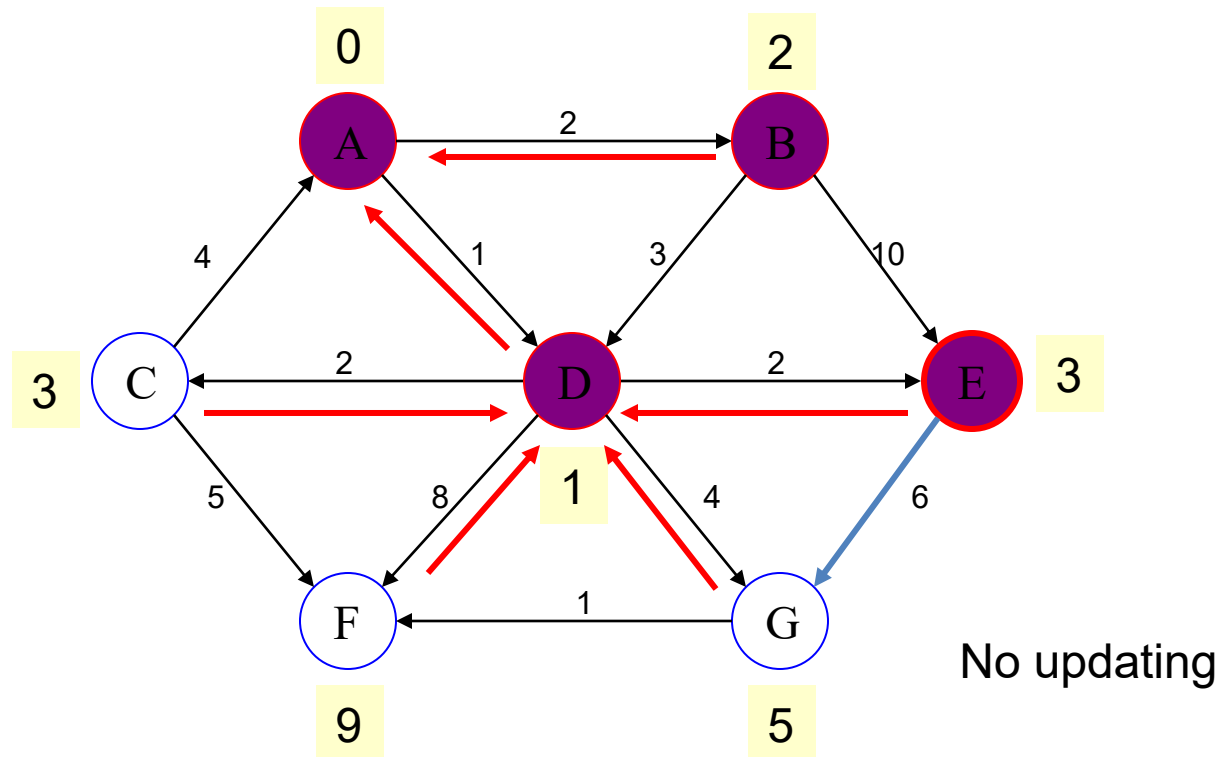
Pick vertex in List with minimum distance (B) and update neighbors



Note : distance(D) not updated since D is already known and distance(E) not updated since it is larger than previously computed

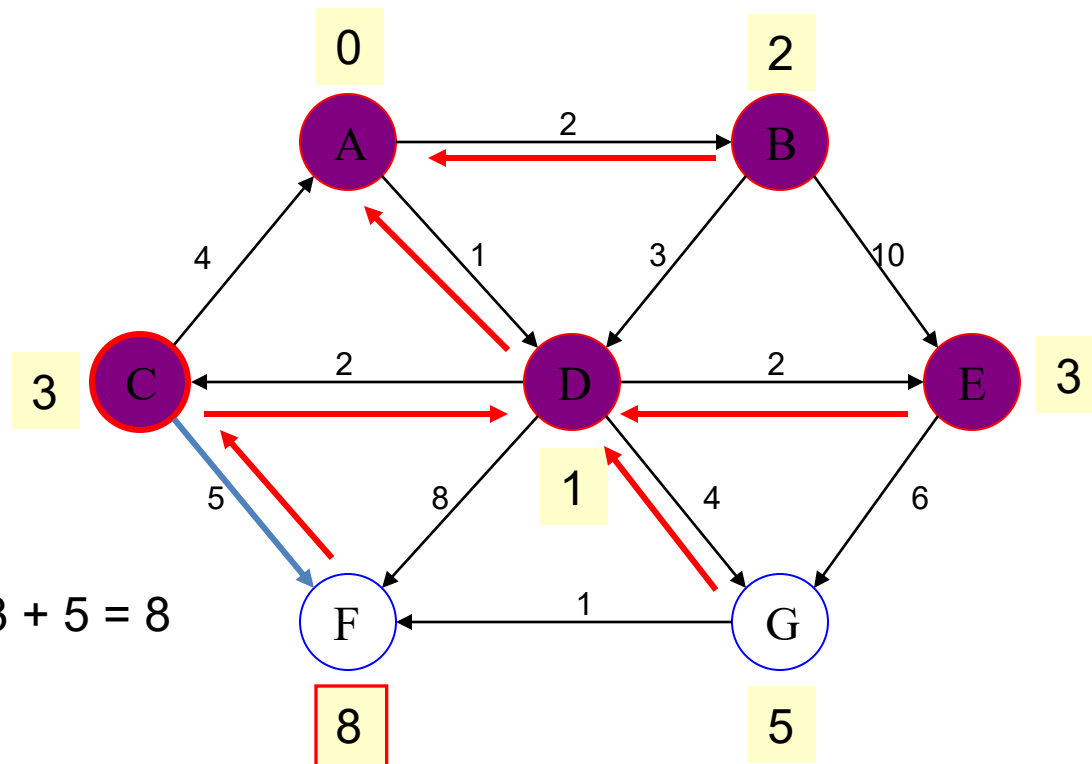
Example: Continued...

Pick vertex List with minimum distance (E) and update neighbors



Example: Continued...

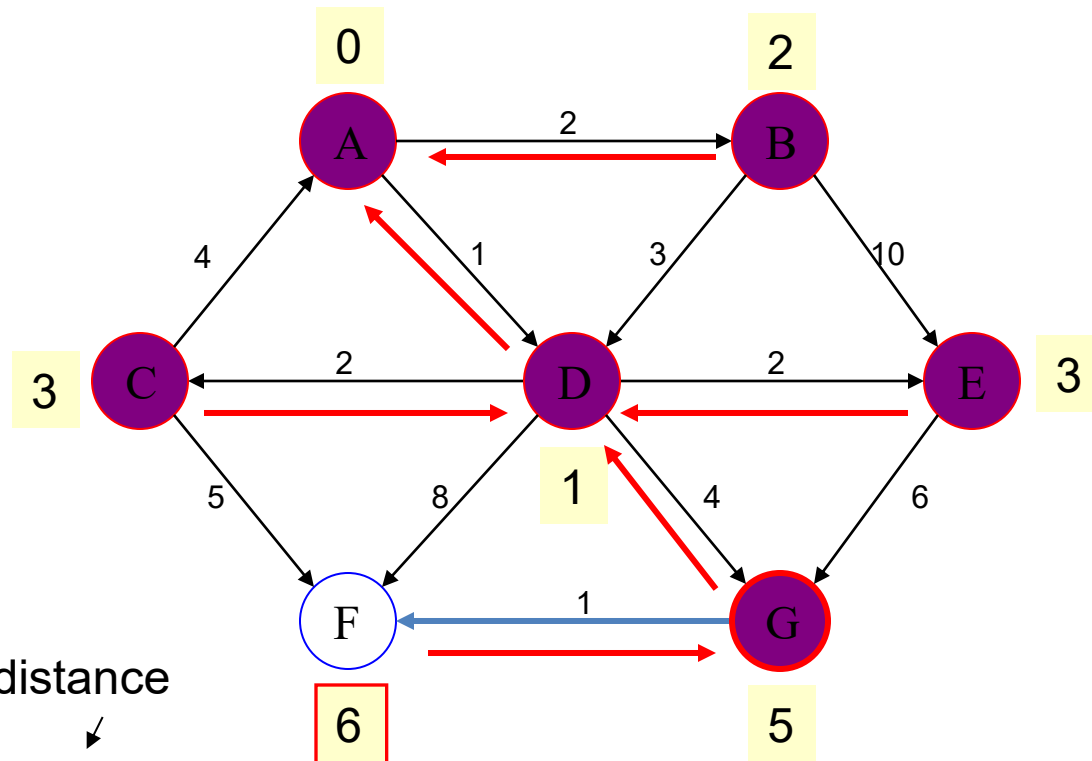
Pick vertex List with minimum distance (C) and update neighbors



$$\text{Distance}(F) = 3 + 5 = 8$$

Example: Continued...

Pick vertex List with minimum distance (G) and update neighbors

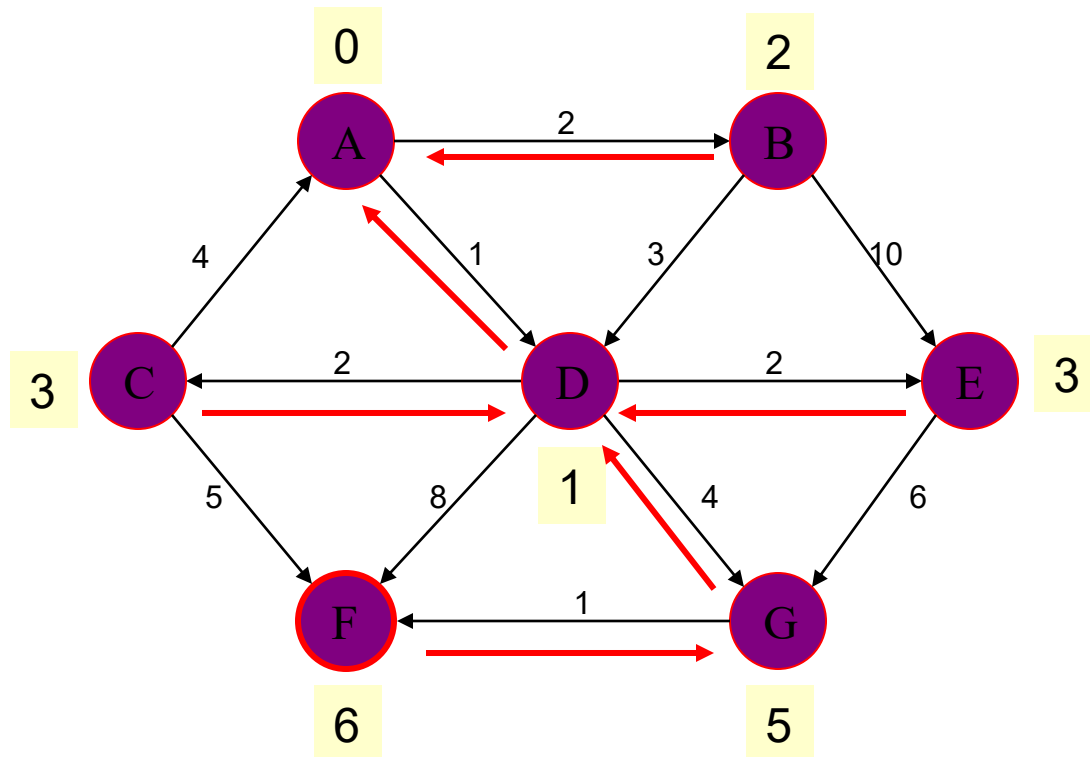


Previous distance



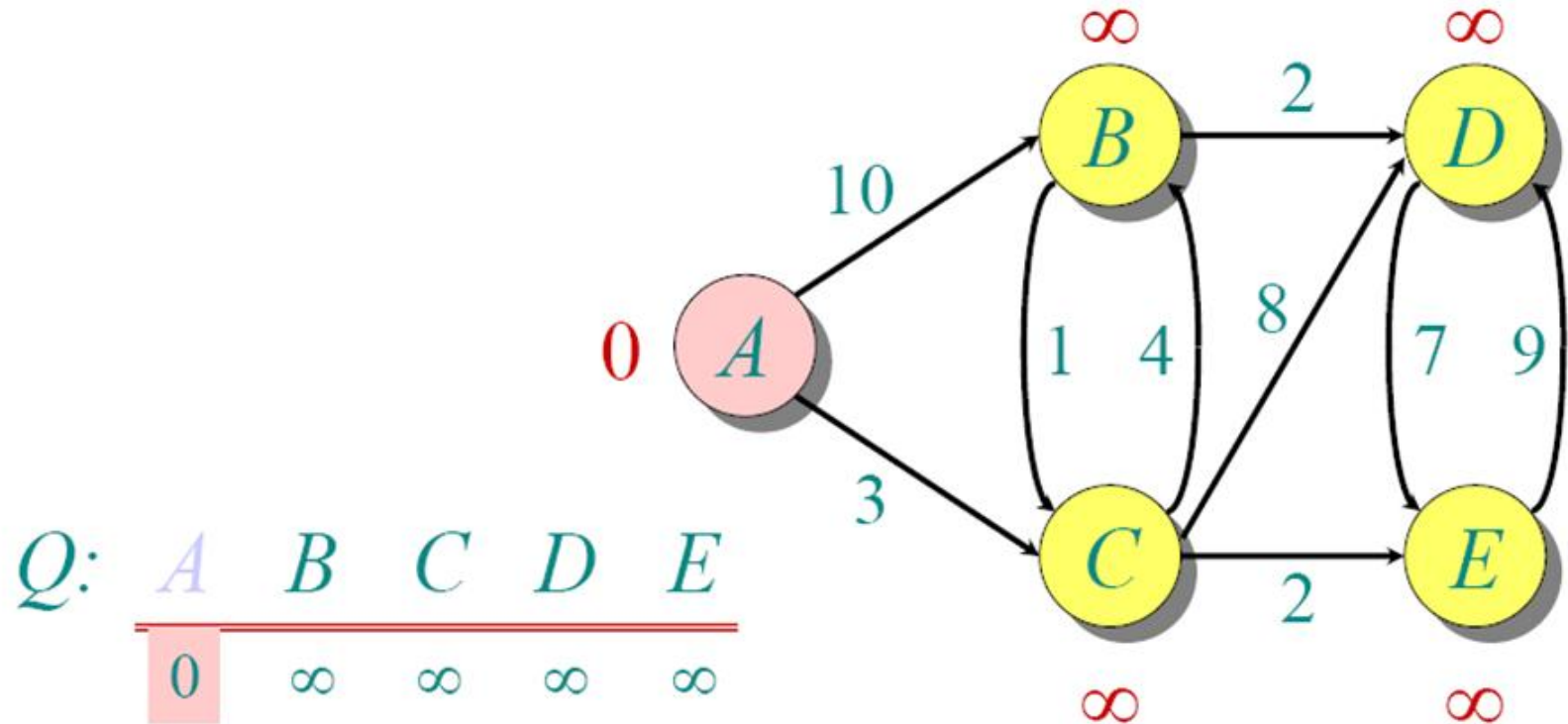
$$\text{Distance}(F) = \min(8, 5+1) = 6$$

Example (end)

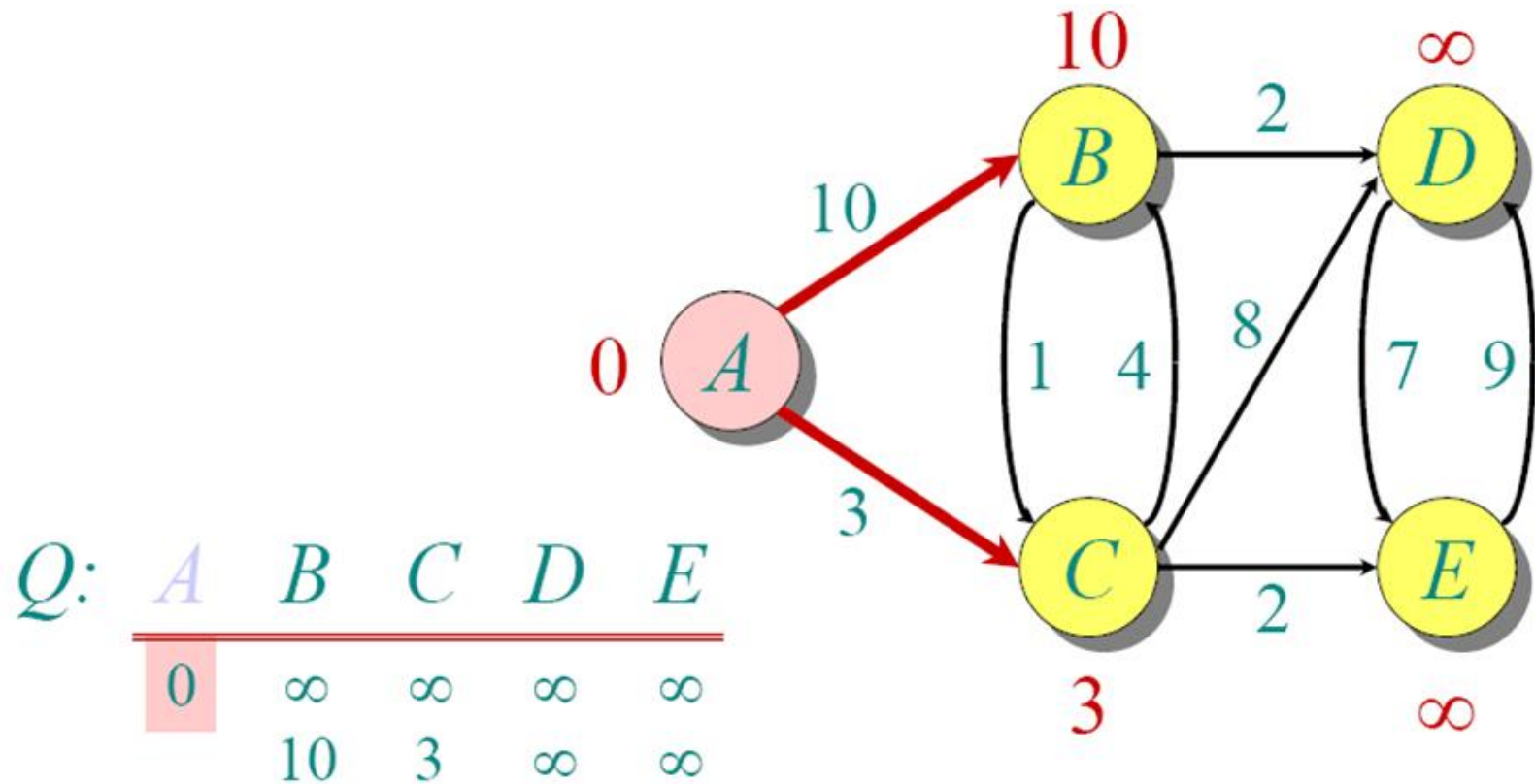


Pick vertex not in S with lowest cost (F) and update neighbors

Another Example

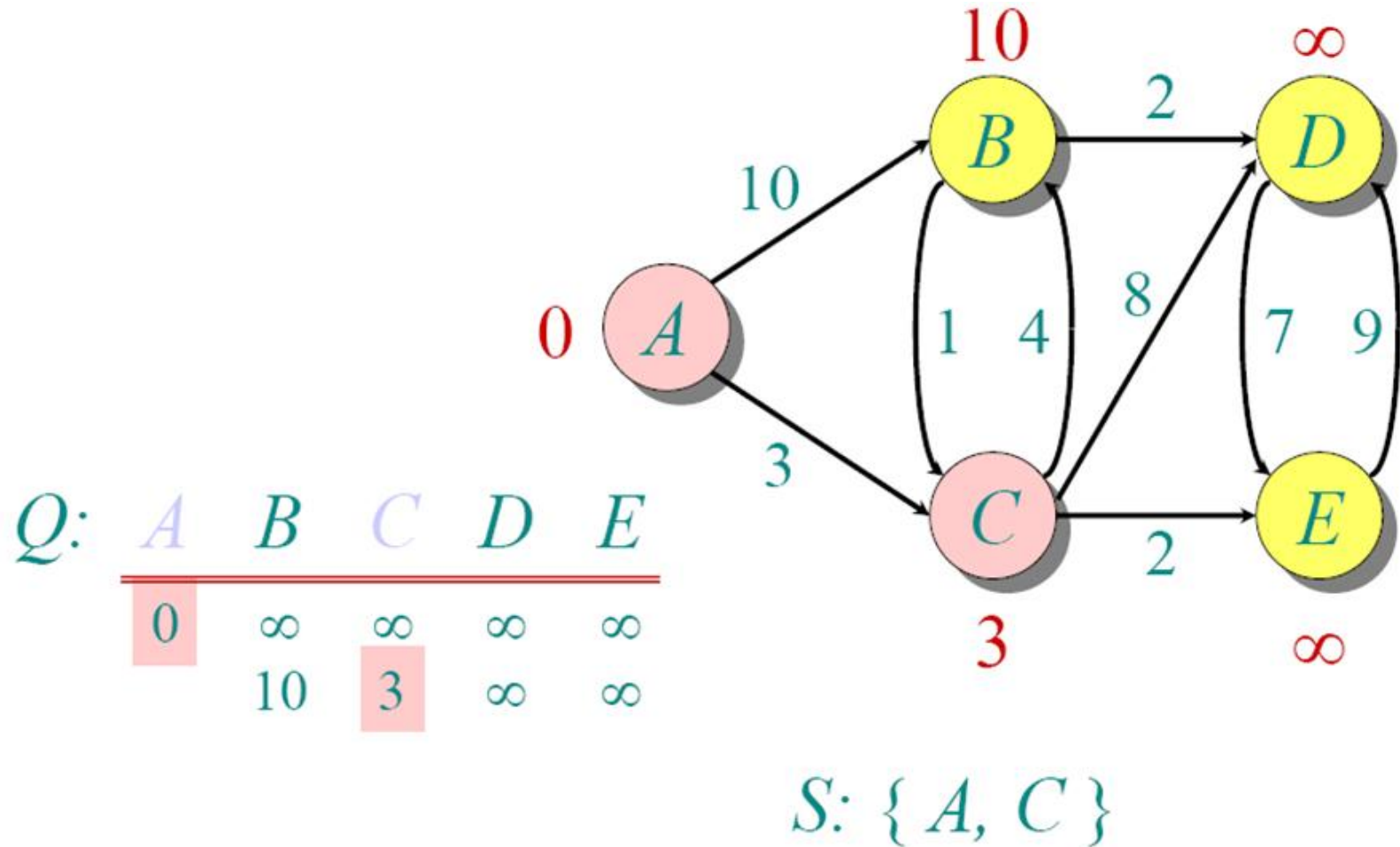


Another Example

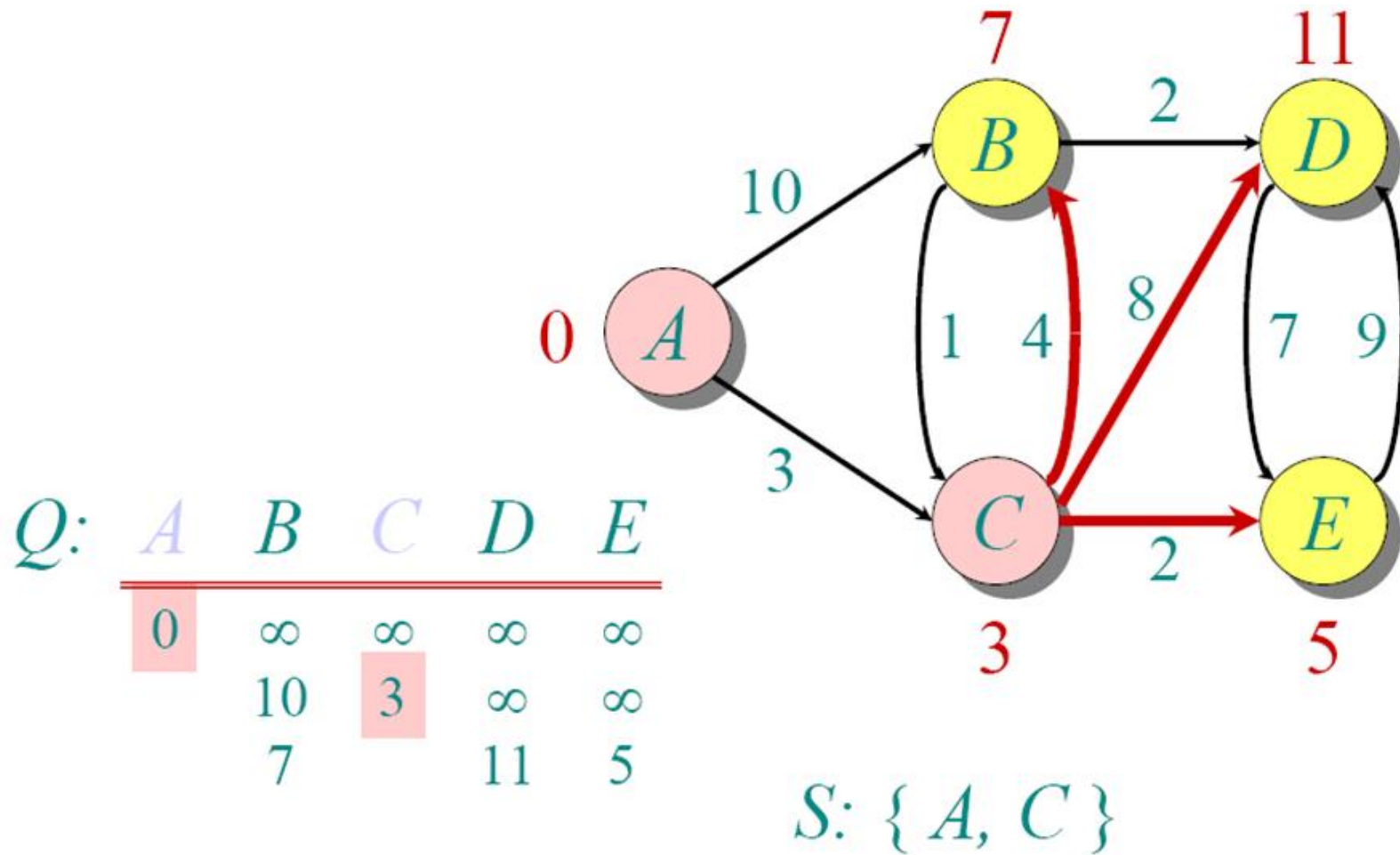


$S: \{ A \}$

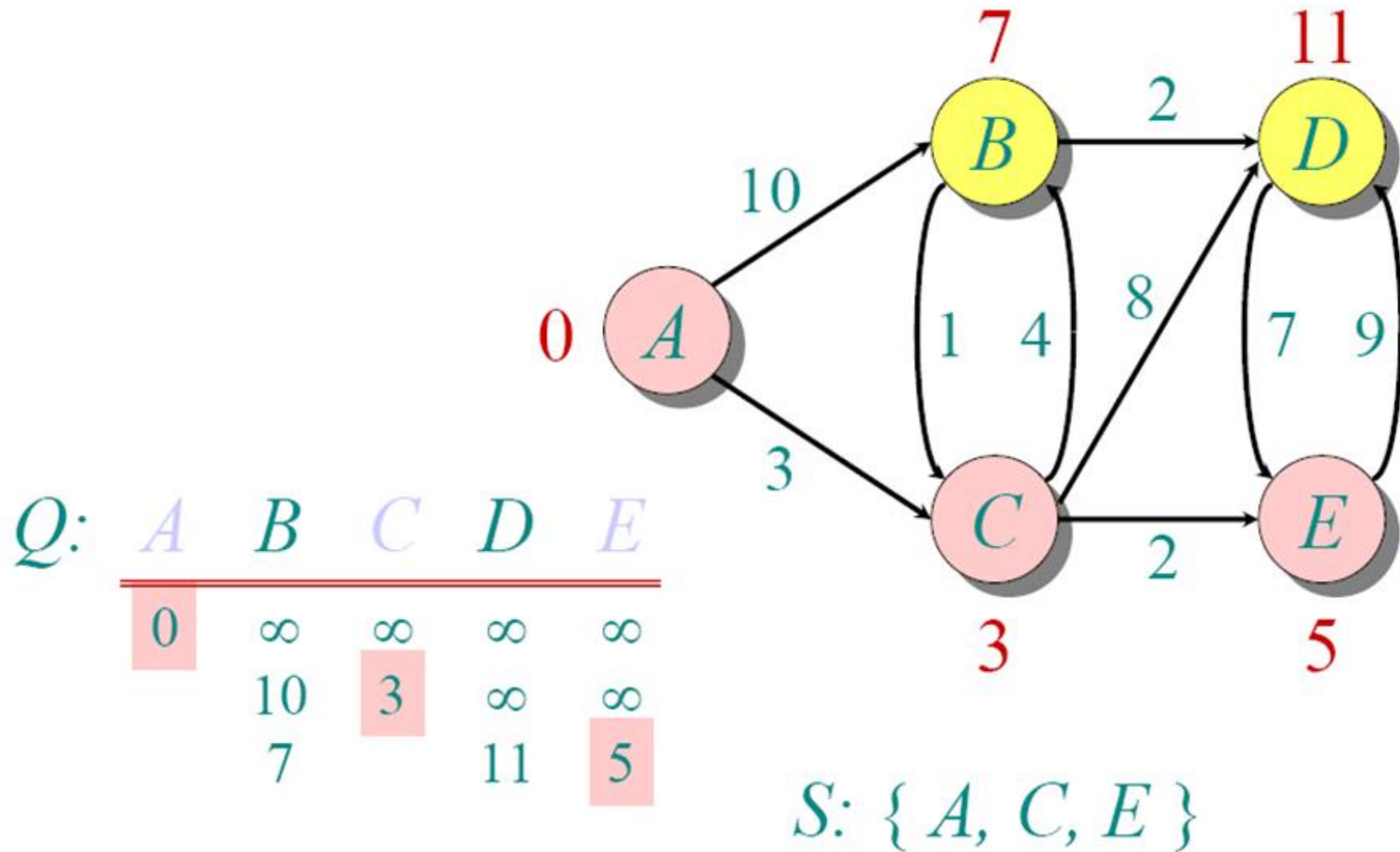
Another Example



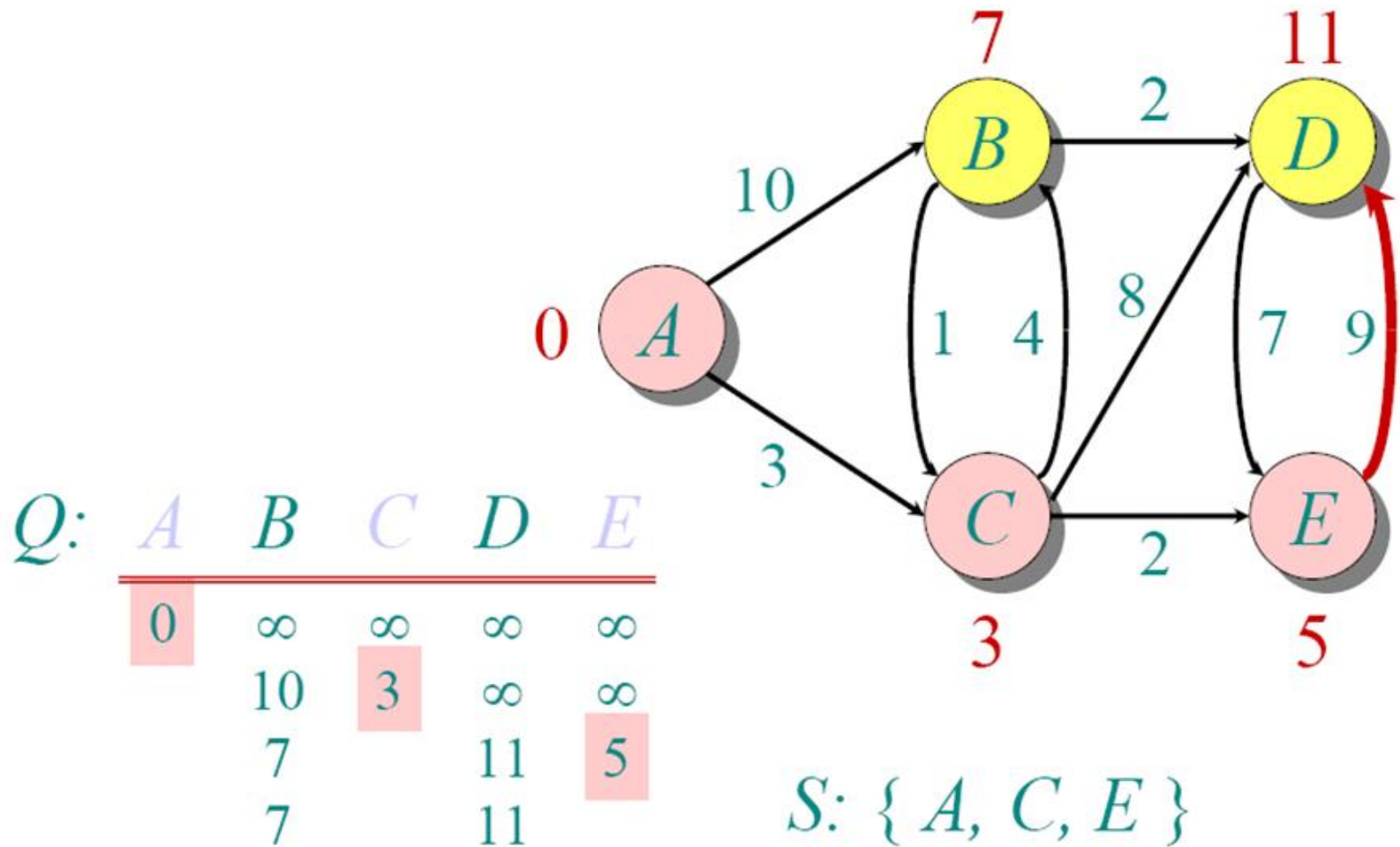
Another Example



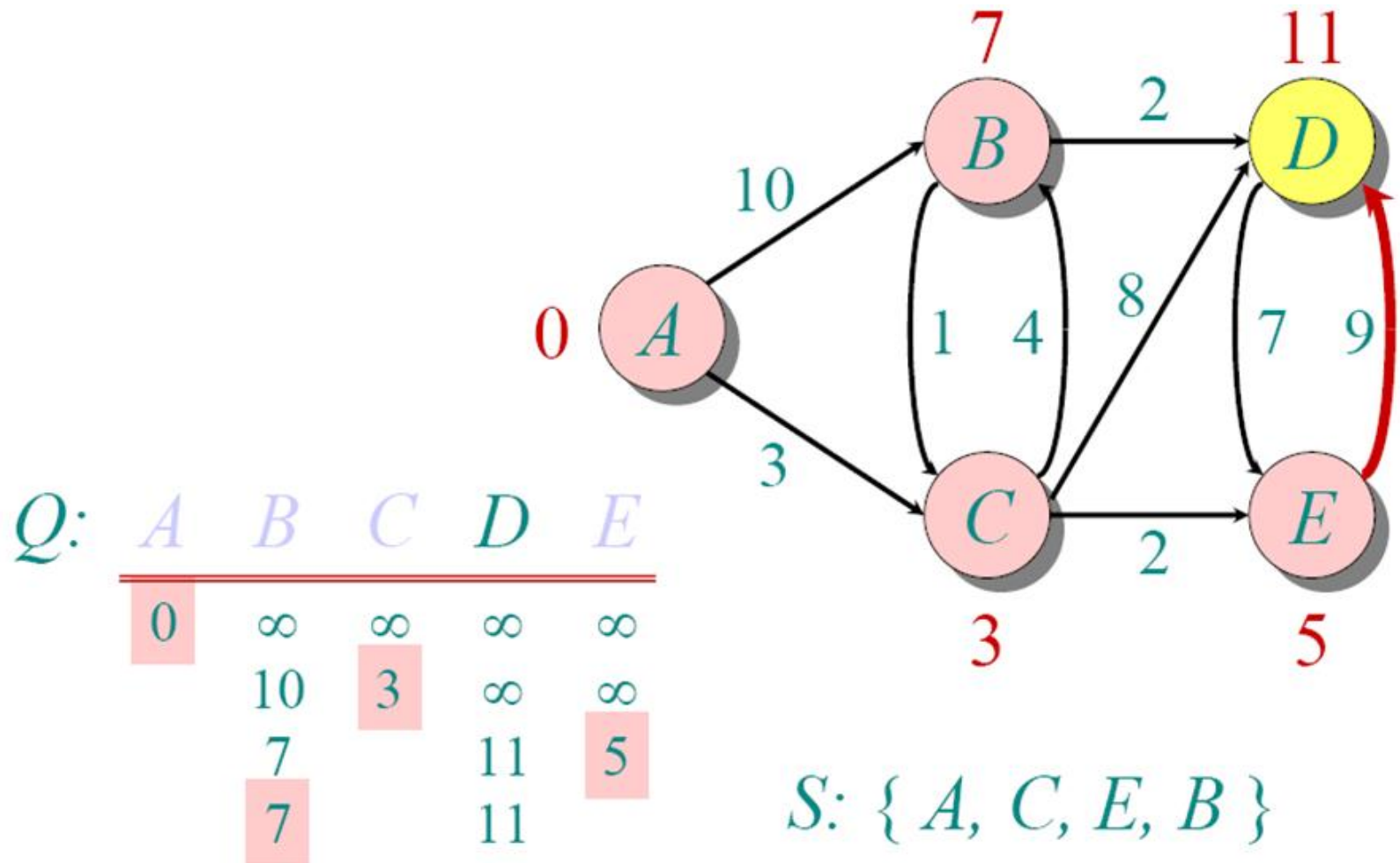
Another Example



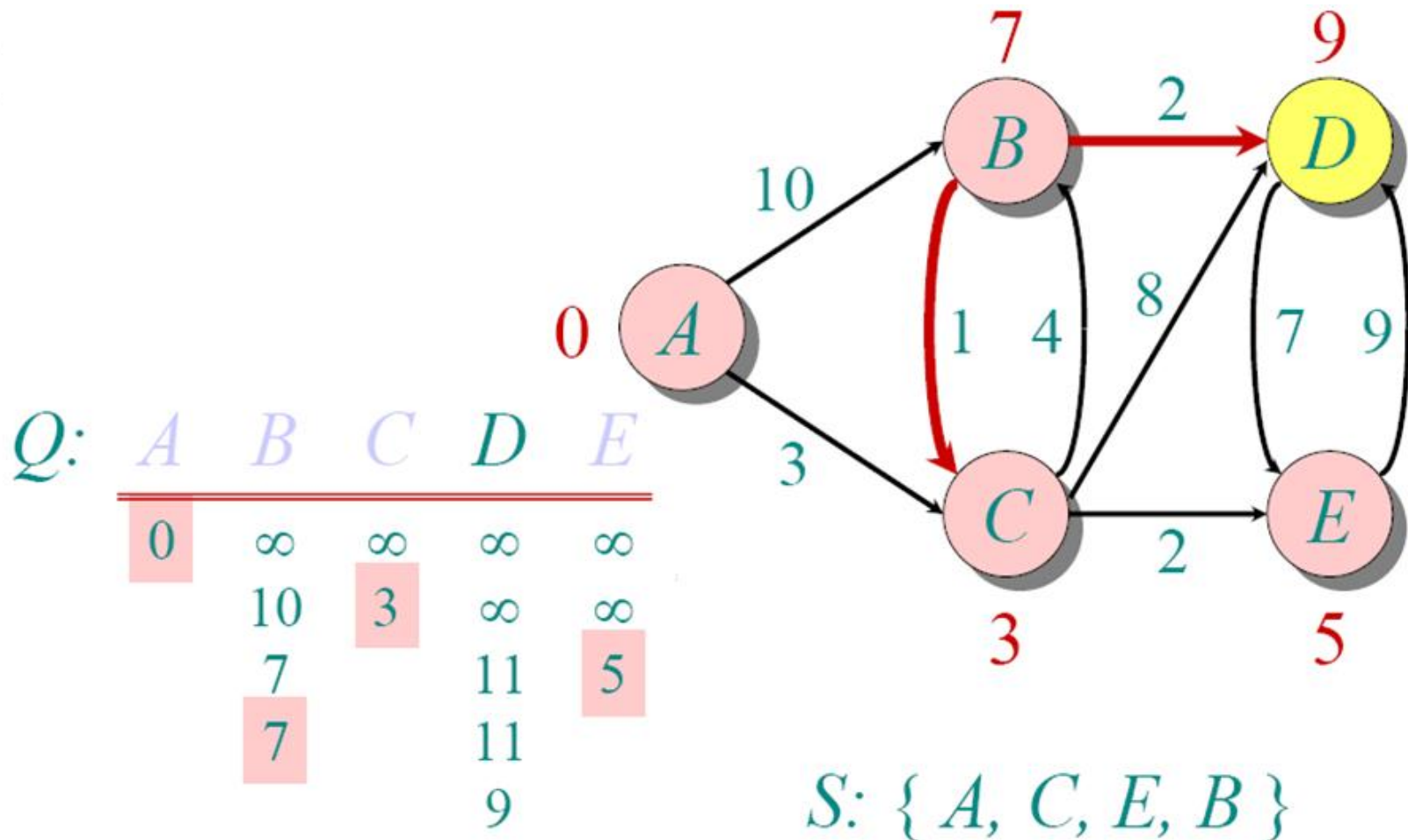
Another Example



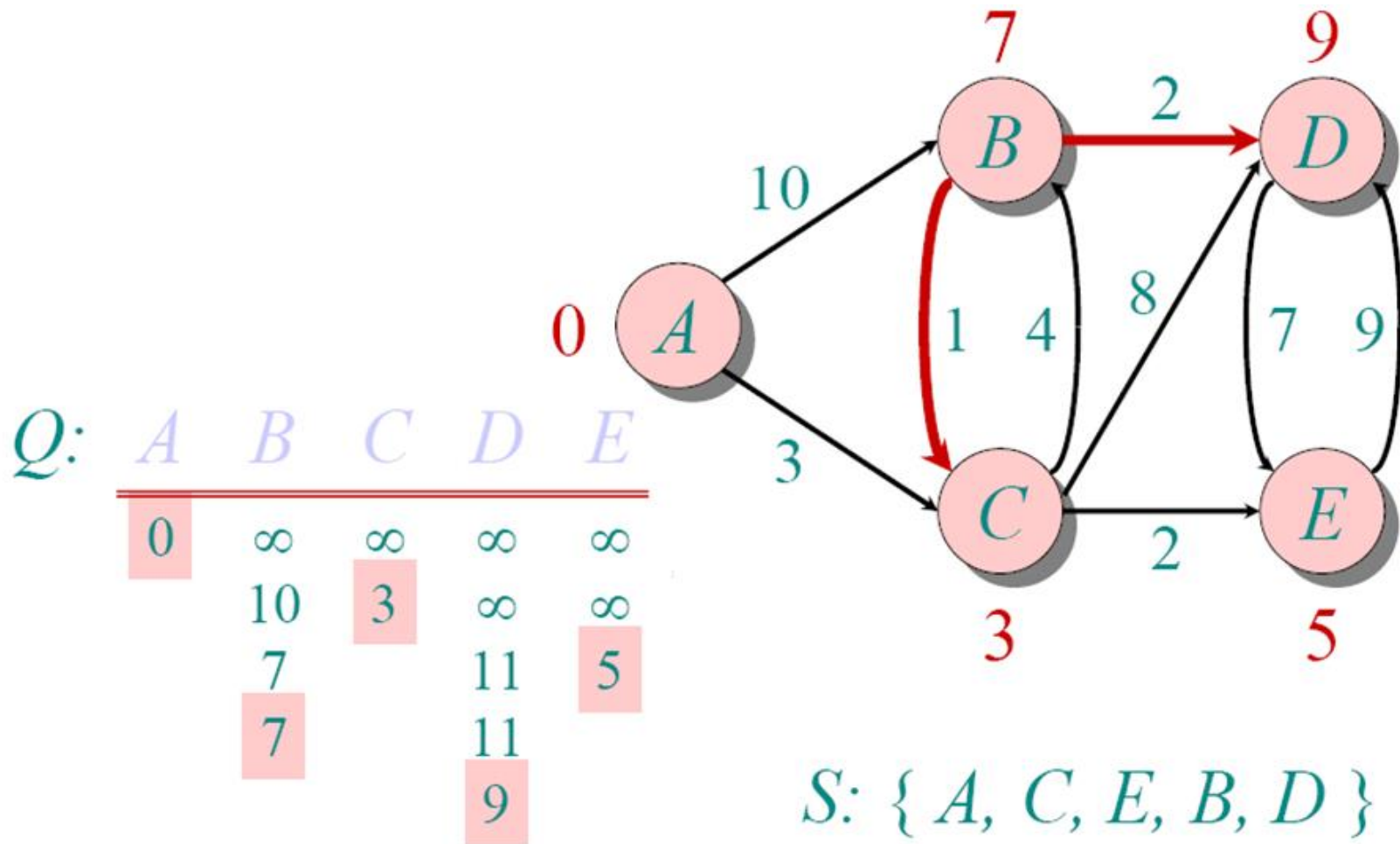
Another Example



Another Example

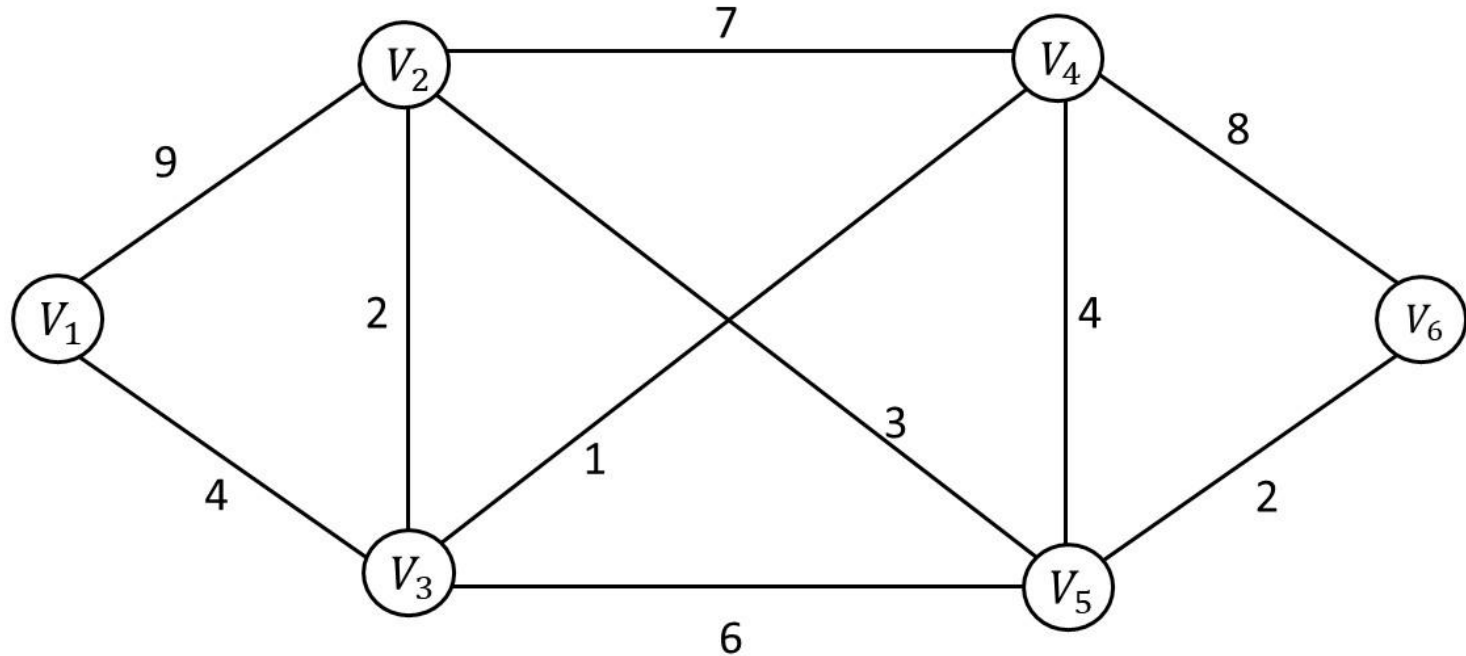


Another Example



Use Dijkstra's Algorithm to find the shortest path the source vertex V_1

Problem 1:



Problem 2:

